

Database Design and ML Prediction Using Leukemia Dataset



A project report submitted as part of the coursework for Database Systems during the 2nd Semester, 2026.

Submitted By

Muhammad Zain

2025-BSCPE-118

Supervised By

Dr. Bilal Ahmad

Spring 2026

B.Sc. Computer Engineering
Department of Electrical, Electronics & Telecom Engineering
University of Engineering and Technology Lahore, Faisalabad Campus

Table of Contents

1. Objectives

2. Introduction / Theory

- 2.1 End-to-End Data Science Pipelines
- 2.2 Database Systems in Healthcare
- 2.3 Machine Learning for Survival Prediction
- 2.4 Normalization and Relational Design

3. Journey to Decide the Dataset

- 3.1 First Attempt: TTS Model
- 3.2 Second Attempt: Popular Datasets
- 3.3 Third Attempt: Leukemia Research Paper Mismatch
- 3.4 Final Decision: TCGA-LAML from cBioPortal

4. Database Design

- 4.1 Entities and Attributes
- 4.2 Entity-Relationship (ER) Diagram
- 4.3 Normalization (1NF, 2NF, 3NF)
- 4.4 MySQL Implementation

5. Fetching the Data (FETCHED.csv)

- 5.1 SQL Queries with JOINS
- 5.2 Dataset Overview

6. Data Preprocessing & Feature Engineering

- 6.1 Dropping Extra Columns
- 6.2 Handling Missing Values
- 6.3 Encoding Categorical Variables
- 6.4 Train-Test Split

7. Model Training

- 7.1 Decision Tree Regressor
- 7.2 Linear Regression (Ridge)
- 7.3 Random Forest Regressor

8. Model Evaluation & Comparison

- 8.1 Evaluation Metrics
- 8.2 Results Table
- 8.3 Analysis
- 8.3 Standalone Inference Test
- 8.4 EDA

9. Inference Script

- 9.1 MLInferenceEngine Class
- 9.2 How the Inference Engine Works

10. Web Application (FastAPI + Dashboard)

- 10.1 Backend (app.py)
- 10.2 Frontend (index.html)
- 10.3 InferenceEngine Class

11. Conclusion

12. References

1.Objectives

- Select and analyze a real-world medical dataset from a credible source (cBioPortal).
- Design and normalize a relational database schema to store the dataset in MySQL.
- Create an Entity-Relationship (ER) diagram representing the database design.
- Implement the designed database in MySQL Workbench using SQL DDL statements.
- Write SQL queries with JOINS to retrieve and export data from the implemented database.
- Perform data preprocessing, feature engineering, and encoding to prepare data for machine learning.
- Train and compare multiple regression models to predict Overall Survival (Months) for AML patients.
- Evaluate models using MAE, MSE, RMSE, and R^2 Score to identify the best-performing algorithm.
- Develop an inference script that can predict survival on new patient records using the trained model.
- Build a FastAPI web application with a responsive dashboard for interactive prediction.
- Demonstrate the integration of database systems with machine learning in a healthcare context.

2.Introduction / Theory

End-to-end data science pipelines combine two major components: a robust database system for storing, managing, and querying structured data, and machine learning models that extract predictive insights from that data. In healthcare, this integration is particularly powerful because clinical datasets are inherently relational — patients belong to departments, have multiple samples, receive treatments, and produce lab results over time. A well-designed relational database ensures data integrity, eliminates redundancy, and enables complex analytical queries that feed directly into ML workflows.

The importance of this integration cannot be overstated in the context of modern healthcare informatics. Hospitals and research institutions generate massive volumes of clinical data daily, from patient demographics and lab results to treatment protocols and survival outcomes. Without a properly designed database, this data would remain scattered across spreadsheets, text files, and paper records, making it impossible to perform the kind of large-scale analysis that leads to meaningful clinical insights. By combining a normalized relational database with machine learning, we create a system where data flows seamlessly from storage to analysis to prediction, enabling healthcare providers to make data-driven decisions that improve patient outcomes.

2.1 End-to-End Data Science Pipelines

A typical end-to-end pipeline follows these stages: (1) data collection and storage in a relational database, (2) data extraction via SQL queries, (3) preprocessing and feature engineering in Python, (4) model training and evaluation, and (5) deployment for inference. This project follows every stage of this pipeline, starting from a raw CSV dataset, normalizing it into 11 MySQL tables, exporting a combined dataset via JOINS, and training regression models to predict patient survival.

Each stage of the pipeline serves a critical purpose in ensuring the quality and reliability of the final predictive model. The database stage establishes a single source of truth where all clinical data is stored consistently with proper constraints and relationships. The extraction stage uses SQL's powerful JOIN operations to combine data from multiple normalized tables into a unified dataset suitable for analysis. The preprocessing stage transforms raw clinical data into a format that machine learning algorithms can process, handling missing values, encoding categorical variables, and selecting relevant features. The training stage applies statistical algorithms to learn patterns in the historical data, and the deployment stage makes these learned patterns available for real-time prediction on new patient records.

2.2 Database Systems in Healthcare

Healthcare datasets like TCGA-LAML (The Cancer Genome Atlas — Acute Myeloid Leukemia) contain rich clinical, genomic, and treatment data for hundreds of patients. Storing this data in a flat CSV file leads to anomalies: update anomalies (changing a doctor's name requires updating multiple rows), insertion anomalies (cannot add a department without a patient), and deletion anomalies (deleting a patient may lose department information). Normalization through 1NF, 2NF, and 3NF eliminates these anomalies by decomposing the data into well-structured relational tables with proper foreign key constraints.

The TCGA-LAML dataset specifically contains information about 200 patients diagnosed with Acute Myeloid Leukemia, a fast-growing cancer of the blood and bone marrow. Each patient record includes demographic information (age, sex, race), clinical measurements (white blood cell count, bone marrow blast percentage), genetic data (cytogenetics, mutation count), treatment details (induction chemotherapy, transplant type), and survival outcomes. Without proper database design, managing this multi-faceted data would be chaotic, leading to data quality issues that compromise both analysis and prediction accuracy.

2.3 Machine Learning for Survival Prediction

Survival prediction is a critical task in oncology. For Acute Myeloid Leukemia (AML), predicting how long a patient will survive based on clinical features (age, WBC count, bone marrow blast percentage, cytogenetics, risk classification) can help clinicians make informed treatment decisions. This project treats survival prediction as a regression problem, where the

target variable is Overall_Survival_Months. Multiple regression models are trained and compared to identify the best approach.

The choice of regression over classification is deliberate in this context. While we could classify patients into survival categories (e.g., short-term vs. long-term survival), regression provides a more granular prediction — the actual number of months a patient is expected to survive. This continuous output is more clinically useful because it allows doctors to discuss specific timeframes with patients and plan treatment schedules accordingly. The regression approach also preserves more information from the original data, as we do not need to arbitrarily bin continuous survival times into discrete categories.

2.4 Normalization and Relational Design

Normalization is a systematic process of organizing relational database tables to minimize data redundancy and dependency. The process involves decomposing a large denormalized table into smaller, well-structured tables linked by foreign keys. In this project, the raw TCGA-LAML CSV file (a flat denormalized format with 54 columns) is decomposed into 11 normalized tables: Patient, Doctor, Department, Sample, Cancer_Type, Clinical, Cytogenetics, Risk, Lab_Results, Treatment, and Outcome. Each table is in Third Normal Form (3NF), ensuring that every non-key attribute depends on the primary key, the whole primary key, and nothing but the primary key.

The normalization process follows a strict methodology. First, we identify all functional dependencies in the denormalized data — relationships where one attribute (or set of attributes) determines the value of another attribute. Then, we decompose the data to eliminate partial dependencies (violations of 2NF) and transitive dependencies (violations of 3NF). For example, in the original flat file, a patient's doctor name and department information would be repeated for every patient in that department, creating redundancy. By separating this into a Doctor table and a Department table, we store each doctor's information exactly once and reference it via foreign keys, ensuring that any update to a doctor's name only needs to be made in one place.

Why We Split Tables (Normalization Benefits)

The original TCGA-LAML CSV file is a single flat file with 54 columns. Storing all this data in one table causes three critical problems:

- **Update Anomaly:** If a doctor's phone number changes, we must update every row where that doctor appears. With 200 patients, this means updating potentially dozens of rows. After normalization, we update the Doctor table once.
- **Insertion Anomaly:** We cannot add a new department to the database unless we also have a patient assigned to that department. After normalization, we can add departments independently.

- **Deletion Anomaly:** If we delete the only patient in a department, we lose all information about that department. After normalization, department information is preserved in the Department table.

By splitting the data into 11 normalized tables (Patient, Doctor, Department, Sample, Cancer_Type, Clinical, Cytogenetics, Risk, Lab_Results, Treatment, Outcome), we ensure that each piece of information is stored exactly once. This eliminates redundancy, prevents anomalies, and makes the database easier to maintain and query.

2.5 Project Flow Diagram

The following diagram shows the high-level flow of our project, from database design to web deployment:

Step 1	Database Design	Design 11 normalized tables in MySQL (3NF)
Step 2	SQL Queries	Write SELECT with JOINs to combine 7 tables into single dataset
Step 3	Data Export	Export combined data to FETCHED.csv (200 rows, 54 columns)
Step 4	Preprocessing	Drop 31 columns, fill missing values, encode categoricals, train/test split
Step 5	ML Training	Train Ridge, Decision Tree, Random Forest on training data
Step 6	Evaluation	Compare models using MAE, MSE, RMSE, R ² Score
Step 7	Inference	Write MLInferenceEngine to predict on new patient records
Step 8	API	Wrap inference in FastAPI POST /predict endpoint
Step 9	Dashboard	Build HTML/CSS/JS frontend with form inputs and model selection

3. Journey to Decide the Dataset

Choosing the right dataset for this project was not a straightforward process. It involved multiple iterations, rejections, and a deepening understanding of what our instructor expected. This section documents the complete journey — from our initial idea to the final selection of the TCGA-LAML dataset from cBioPortal.

3.1 First Attempt: TTS Model

When our instructor first announced the project, he gave us complete freedom: “You can train anything.” Excited by this open-ended opportunity, our group decided to build a Text-to-Speech (TTS) model. We believed this would be an impressive and technically challenging project. We began researching TTS architectures, looked into models like Tacotron and FastSpeech, and started searching for suitable speech datasets.

However, when we presented our idea to Dr. Bilal Ahmad, he immediately raised a concern: the project needed to involve a database component. A TTS model, while interesting from a deep learning perspective, did not naturally fit into a database systems course. There was no relational structure to model, no SQL queries to write, and no normalization to perform. The dataset for TTS would simply be audio files and their transcriptions — a flat, non-relational structure. We understood the concern and went back to the drawing board.

3.2 Second Attempt: Popular Datasets

After the TTS rejection, we shifted our focus to datasets that could work for both machine learning and database design. I was at home, guiding my friend and groupmate over voice call, as we scrolled through Kaggle, UCI Machine Learning Repository, and Google Dataset Search. We needed a dataset that had enough relational structure to normalize into multiple tables, while also having a clear prediction target for ML.

We selected five datasets and presented them to our instructor. However, Dr. Ahmad rejected them all with a specific reason: “These are very popular datasets. I think other students have already chosen them.” He opened cBioPortal on his screen and pointed to a leukemia dataset, saying, “Show me its research paper.” This was a pivotal moment — our instructor wanted us to work with a dataset that was not only technically suitable but also had genuine research credibility.

3.3 Third Attempt: Leukemia Research Paper Mismatch

Following our instructor’s suggestion, we explored the leukemia datasets available on cBioPortal. We found a dataset that seemed promising and began examining it. When we looked up its associated research paper, we discovered a mismatch — the dataset we had selected did not align with the research paper that was supposedly associated with it. The columns, patient counts, or clinical attributes did not match what was described in the publication. This was a significant setback, as using a dataset without proper research backing would undermine the credibility of our project.

We sat down again, this time being more careful. We cross-referenced datasets with their publications, verified that the clinical attributes matched the described methodology, and ensured that the dataset had sufficient size and complexity for both normalization and ML training.

3.4 Final Decision: TCGA-LAML from cBioPortal

After careful research, we found the TCGA-LAML (The Cancer Genome Atlas — Acute Myeloid Leukemia) dataset on cBioPortal (Memorial Sloan Kettering Cancer Center). The dataset is publicly available at https://www.cbioportal.org/study?id=laml_tcga_pub and is associated with the publication: “Genomic and Epigenomic Landscapes of Adult De Novo Acute

Myeloid Leukemia” published in the New England Journal of Medicine, 2013 (PMID: 23634996).

Key characteristics of the dataset:

Property	Value
Dataset Name	TCGA-LAML (Acute Myeloid Leukemia)
Source	National Cancer Institute, USA (cancer.gov/tcga)
Published In	New England Journal of Medicine, 2013 (PMID: 23634996)
Hosted On	cBioPortal — Memorial Sloan Kettering Cancer Center
Study ID	laml_tcga_pub
Patients	200 Real AML Patients
Features	35 Clinical Attributes
Target	Overall Survival (Months) — Regression Task
Access	Publicly Free — https://www.cbioportal.org/study?id=laml_tcga_pub

This dataset was ideal because it had a clear relational structure (patients, doctors, departments, clinical records, lab results, treatments, outcomes), a well-defined prediction target (overall survival in months), and strong research backing from a top-tier medical journal. Dr. Ahmad approved this selection, and we proceeded with the project.

4. Database Design

4.1 Entities and Attributes

The raw TCGA-LAML CSV file is a flat denormalized file with 54 columns. To design a proper relational schema, we identified the core entities and their attributes from the dataset. Each entity represents a distinct concept in the healthcare domain:

- Patient — Core demographic information: Patient_ID, Study_ID, Diagnosis_Age, Sex, Race_Category, Number_of_Samples_Per_Patient, Doctor_ID, Cancer_Type_ID.
- Doctor — Assigned doctor: Doctor_ID, Doctor_Name, Specialization, Email, Phone, Department_ID.
- Department — Hospital department: Department_ID, Department_Name, Location, Head_of_Department, Contact_Number.
- Sample — Biological sample: Sample_ID, Patient_ID, Sample_Type.
- Cancer_Type — Cancer classification: Cancer_Type_ID, Cancer_Type, Cancer_Type_Detailed, Oncotree_Code.
- Clinical — Assessment data: Clinical_ID, Patient_ID, FAB_Code, Tumor_Other_Histologic_Subtype, FAB_Description, Cytogenetic_ID, Risk_ID.
- Cytogenetics — Cytogenetic profile: Cytogenetic_ID, Cytogenetics, Cytogenetic_Code_Other, DZ_Stat.

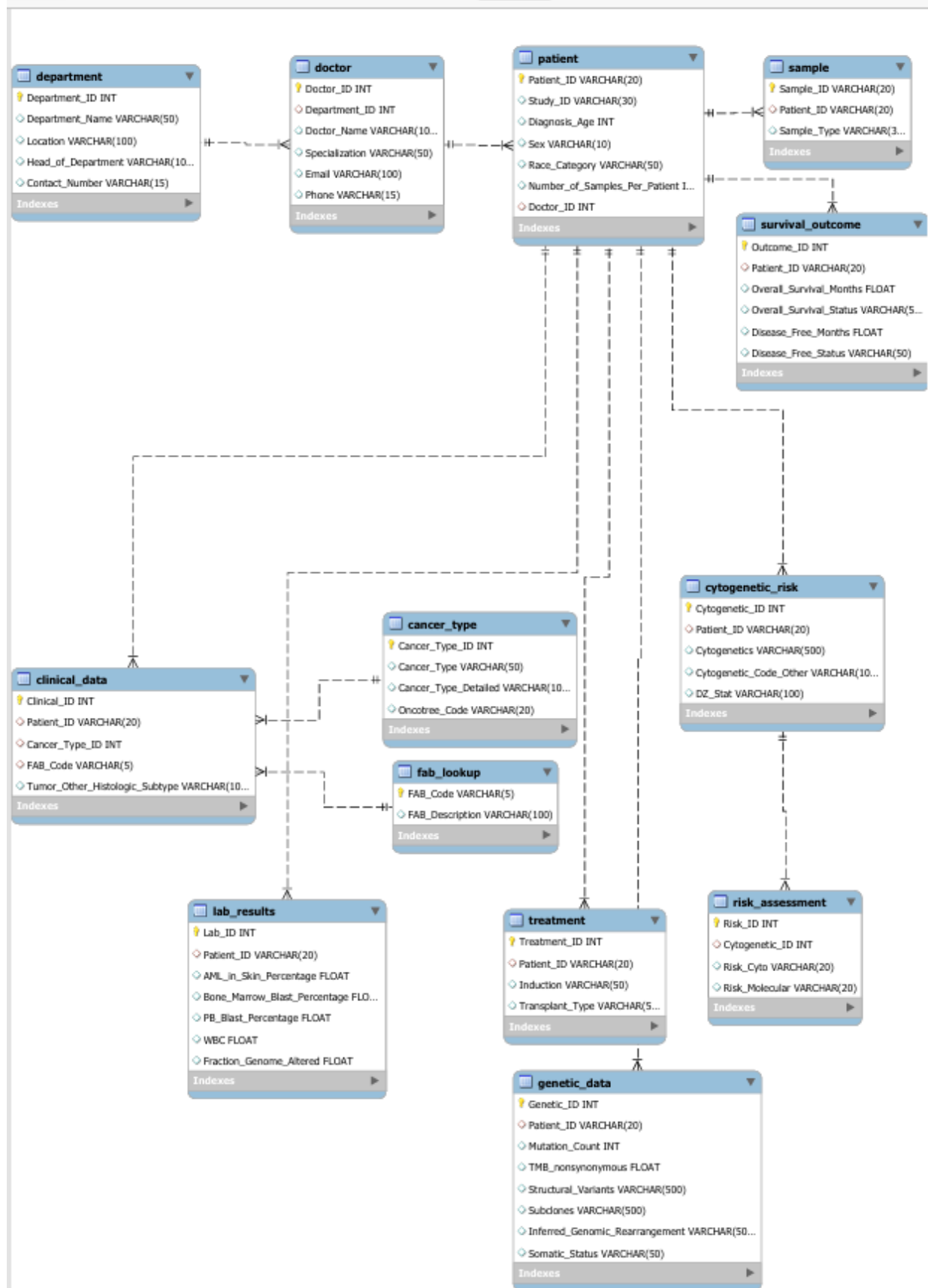
- Risk — Risk classification: Risk_ID, Risk_Cyto, Risk_Molecular.
- Lab_Results — Laboratory and genomic measurements: Lab_ID, Patient_ID, AML_in_Skin_Percentage, Bone_Marrow_Blast_Percentage, PB_Blast_Percentage, WBC, Fraction_Genome_Altered, Genetic_ID, Mutation_Count, TMB_nonsynonymous, Structural_Variants, Subclones, Inferred_Genomic_Rearrangement, Somatic_Status.
- Treatment — Treatment details: Treatment_ID, Patient_ID, Induction, Transplant_Type.
- Outcome — Survival outcomes: Outcome_ID, Patient_ID, Overall_Survival_Months, Overall_Survival_Status, Disease_Free_Months, Disease_Free_Status.

4.2 Entity-Relationship (ER) Diagram

The Entity-Relationship (ER) diagram below represents the designed relational schema. Each box represents an entity (table), and the lines represent foreign key relationships between entities.

- Patient (1) — (M) Doctor: Each patient is assigned to one doctor, but a doctor can have many patients.
- Doctor (M) — (1) Department: Many doctors belong to one department.
- Patient (1) — (M) Sample: Each patient can have multiple biological samples collected.
- Patient (1) — (1) Clinical: Each patient has one clinical assessment record.
- Clinical (M) — (1) Cytogenetics: Many clinical records can reference the same cytogenetic profile.
- Clinical (M) — (1) Risk: Many clinical records share the same risk classification.
- Patient (1) — (1) Lab_Results: Each patient has one set of laboratory measurements.
- Patient (1) — (1) Treatment: Each patient has one treatment record.
- Patient (1) — (1) Outcome: Each patient has one survival outcome record.
- Patient (M) — (1) Cancer_Type: Many patients can have the same cancer type.

The following diagram shows all 11 entities and their foreign key relationships:



Foreign Key Relationships:

- Patient (1) — (M) Doctor: Each patient is assigned to one doctor, but a doctor can have many patients.
- Doctor (M) — (1) Department: Many doctors belong to one department.
- Patient (1) — (M) Sample: Each patient can have multiple biological samples collected.
- Patient (1) — (1) Clinical: Each patient has one clinical assessment record.
- Clinical (M) — (1) Cytogenetics: Many clinical records can reference the same cytogenetic profile.
- Clinical (M) — (1) Risk: Many clinical records share the same risk classification.
- Patient (1) — (1) Lab_Results: Each patient has one set of laboratory measurements.
- Patient (1) — (1) Treatment: Each patient has one treatment record.
- Patient (1) — (1) Outcome: Each patient has one survival outcome record.
- Patient (M) — (1) Cancer_Type: Many patients can have the same cancer type.

4.3 Normalization (1NF, 2NF, 3NF)

First Normal Form (1NF)

The raw TCGA-LAML CSV already contains atomic values in each cell — there are no multi-valued attributes or repeating groups. Each column holds a single value, and each row represents a single record. Therefore, the dataset is already in First Normal Form.

Second Normal Form (2NF)

Since all primary keys in this schema are single-column surrogate keys (e.g., Patient_ID, Doctor_ID, Clinical_ID), there are no partial dependencies. Every non-key attribute depends on the entire primary key of its respective table. The dataset satisfies 2NF.

Third Normal Form (3NF)

Transitive dependencies were identified and eliminated during decomposition. The following transitive dependencies were resolved:

- Patient_ID → Doctor_ID → Department_ID → Department_Name: Doctor information and department details were separated into their own tables.
- Clinical_ID → Cytogenetic_ID → Cytogenetics, DZ_Stat: Cytogenetic data was moved to a separate Cytogenetics table.
- Clinical_ID → Risk_ID → Risk_Cyto, Risk_Molecular: Risk classification data was separated into a Risk table.
- Cancer_Type_ID → Cancer_Type, Cancer_Type_Detailed, Oncotree_Code: Cancer type details were moved to a Cancer_Type table.

After 3NF decomposition, every non-key attribute in every table depends on the primary key, the whole primary key, and nothing but the primary key. The schema is fully normalized.

4.4 MySQL Implementation

The following SQL code creates the TCGA_LAML database and all 11 normalized tables with appropriate data types, primary keys, and foreign key constraints.

SQL

```
CREATE DATABASE IF NOT EXISTS TCGA_LAML;  
USE TCGA_LAML;
```

Table 1: Department

SQL

```
CREATE TABLE Department (  
    Department_ID      INT PRIMARY KEY,  
    Department_Name    VARCHAR() NOT NULL,  
    Location           VARCHAR(),  
    Head_of_Department VARCHAR(),  
    Contact_Number     VARCHAR()  
);
```

Table 2: Doctor

SQL

```
CREATE TABLE Doctor (  
    Doctor_ID          INT PRIMARY KEY,  
    Doctor_Name        VARCHAR() NOT NULL,  
    Specialization     VARCHAR(),  
    Email              VARCHAR(),  
    Phone              VARCHAR(),  
    Department_ID      INT,  
    FOREIGN KEY (Department_ID) REFERENCES Department(Department_ID)  
);
```

Table 3: Cancer_Type

SQL

```
CREATE TABLE Cancer_Type (  
    Cancer_Type_ID      INT PRIMARY KEY,  
    Cancer_Type         VARCHAR(),  
    Cancer_Type_Detailed VARCHAR(),  
    Oncotree_Code       VARCHAR()  
);
```

Table 4: Patient

SQL

```
CREATE TABLE Patient (  
    Patient_ID          VARCHAR() PRIMARY KEY,  
    Study_ID            VARCHAR(),  
    Diagnosis_Age       INT,  
    Sex                 VARCHAR(),  
    Race_Category       VARCHAR(),  
    Number_of_Samples_Per_Patient INT,  
    Doctor_ID           INT,  
    Cancer_Type_ID      INT,  
    FOREIGN KEY (Doctor_ID) REFERENCES Doctor(Doctor_ID),  
    FOREIGN KEY (Cancer_Type_ID) REFERENCES Cancer_Type(Cancer_Type_ID)  
);
```

Table 5: Sample

SQL

```
CREATE TABLE Sample (  
    Sample_ID          VARCHAR() PRIMARY KEY,  
    Patient_ID         VARCHAR(),  
    Sample_Type        VARCHAR(),  
    FOREIGN KEY (Patient_ID) REFERENCES Patient(Patient_ID)  
);
```

Table 6: Cytogenetics

SQL

```
CREATE TABLE Cytogenetics (  
    Cytogenetic_ID      INT PRIMARY KEY,  
    Cytogenetics        VARCHAR(),  
    Cytogenetic_Code_Other VARCHAR(),  
    DZ_Stat             VARCHAR()  
);
```

Table 7: Risk

SQL

```
CREATE TABLE Risk (  
    Risk_ID      INT PRIMARY KEY,  
    Risk_Cyto    VARCHAR(),  
    Risk_Molecular VARCHAR()  
);
```

Table 8: Clinical

SQL

```
CREATE TABLE Clinical (  
    Clinical_ID      INT PRIMARY KEY,  
    Patient_ID       VARCHAR(),  
    FAB_Code         VARCHAR(),  
    Tumor_Other_Histologic_Subtype VARCHAR(),  
    FAB_Description   VARCHAR(),  
    Cytogenetic_ID   INT,  
    Risk_ID          INT,  
    FOREIGN KEY (Patient_ID) REFERENCES Patient(Patient_ID),  
    FOREIGN KEY (Cytogenetic_ID) REFERENCES Cytogenetics(Cytogenetic_ID),  
    FOREIGN KEY (Risk_ID) REFERENCES Risk(Risk_ID)  
);
```

Table 9: Lab_Results

SQL

```
CREATE TABLE Lab_Results (  
    Lab_Results_ID      INT PRIMARY KEY,  
    Lab_Results         VARCHAR(),  
    Lab_Results_Code_Other VARCHAR(),  
    Lab_Results_Stat     VARCHAR()  
);
```

```
Lab_ID                INT PRIMARY KEY,  
Patient_ID            VARCHAR(),  
AML_in_Skin_Percentage  FLOAT,  
Bone_Marrow_Blast_Percentage  INT,  
PB_Blast_Percentage    FLOAT,  
WBC                   FLOAT,  
Fraction_Genome_Altered  FLOAT,  
Mutation_Count         INT,  
TMB_nonsynonymous       FLOAT,  
Structural_Variants     FLOAT,  
Subclones              VARCHAR(),  
Inferred_Genomic_Rearrangement  VARCHAR(),  
Somatic_Status         VARCHAR(),  
FOREIGN KEY (Patient_ID) REFERENCES Patient(Patient_ID)  
);
```

Table 10: Treatment

SQL

```
CREATE TABLE Treatment (  
    Treatment_ID    INT PRIMARY KEY,  
    Patient_ID      VARCHAR(),  
    Induction        VARCHAR(),  
    Transplant_Type  VARCHAR(),  
    FOREIGN KEY (Patient_ID) REFERENCES Patient(Patient_ID)  
);
```

Table 11: Outcome

SQL

```
CREATE TABLE Outcome (  
    Outcome_ID      INT PRIMARY KEY,  
    Patient_ID      VARCHAR(),  
    Overall_Survival_Months  FLOAT,  
    Overall_Survival_Status  VARCHAR(),  
    Disease_Free_Months      FLOAT,  
    Disease_Free_Status      VARCHAR(),  
    FOREIGN KEY (Patient_ID) REFERENCES Patient(Patient_ID)  
);
```


5. Fetching the Data (FETCHED.csv)

5.1 SQL Queries with JOINS

After creating the database and populating all 11 tables, we wrote SQL queries using SELECT with multiple JOINS to export a combined dataset for machine learning. The following query retrieves all relevant clinical, genomic, treatment, and outcome data into a single result set.

SQL

```
SELECT
    p.Patient_ID, p.Diagnosis_Age, p.Sex, p.Race_Category,
    c.FAB_Code, cy.Cytogenetics, cy.DZ_Stat,
    r.Risk_Cyto, r.Risk_Molecular,
    l.Bone_Marrow_Blast_Percentage, l.PB_Blast_Percentage,
    l.WBC, l.Fraction_Genome_Altered, l.Mutation_Count,
    l.TMB_nonsynonymous, l.Structural_Variants, l.Subclones,
    l.Inferred_Genomic_Rearrangement,
    t.Induction, t.Transplant_Type,
    o.Overall_Survival_Months
FROM Patient p
JOIN Clinical c      ON p.Patient_ID = c.Patient_ID
JOIN Cytogenetics cy ON c.Cytogenetic_ID = cy.Cytogenetic_ID
JOIN Risk r          ON c.Risk_ID = r.Risk_ID
JOIN Lab_Results l    ON p.Patient_ID = l.Patient_ID
JOIN Treatment t      ON p.Patient_ID = t.Patient_ID
JOIN Outcome o        ON p.Patient_ID = o.Patient_ID
WHERE o.Overall_Survival_Months IS NOT NULL
ORDER BY p.Patient_ID;
```

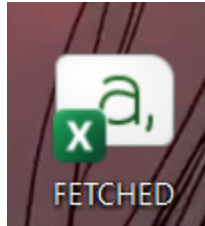
The combined data was exported to CSV using MySQL's INTO OUTFILE command:

SQL

```
SELECT p.Patient_ID, p.Diagnosis_Age, p.Sex, p.Race_Category,
    c.FAB_Code, cy.DZ_Stat, r.Risk_Cyto, r.Risk_Molecular,
    l.Bone_Marrow_Blast_Percentage, l.WBC, l.Mutation_Count,
    t.Induction, t.Transplant_Type, o.Overall_Survival_Months
INTO OUTFILE 'C:/exported_data/FETCHED.csv'
FIELDS TERMINATED BY ',' ENCLOSED BY ''
LINES TERMINATED BY '\n';
```

5.2 Dataset Overview

The exported FETCHED.csv contains 200 patient records with 54 columns. Each row represents a single AML patient with their complete clinical profile, lab results, treatment details, and survival outcome. The target variable for our ML models is Overall_Survival_Months, which ranges from 0.2 months to over 118 months.



6. Data Preprocessing & Feature Engineering

Before training any machine learning model, the raw FETCHED.csv must be cleaned, transformed, and prepared. This section describes each preprocessing step in detail. Data preprocessing is often the most time-consuming phase of a machine learning project, typically consuming 60-80% of the total project time. The quality of the preprocessing directly impacts model performance, as even the most sophisticated algorithm cannot compensate for poorly prepared data. In the context of clinical data, preprocessing is particularly critical because missing values, inconsistent formats, and outliers can have serious implications for model accuracy and clinical utility.

6.1 Dropping Extra Columns

The FETCHED.csv contains many columns that are artifacts of the database structure (doctor names, department details, cancer type metadata) or that would cause data leakage (Overall_Survival_Status, Disease_Free_Months). These columns must be removed before training.

Code

```
# Drop extra database-project columns

extra_cols = [

    'Doctor_ID', 'Doctor_Name', 'Specialization', 'Email', 'Phone',

    'Department_ID', 'Department_Name', 'Location', 'Head_of_Department',
```

```
'Contact_Number', 'Cancer_Type_ID', 'Clinical_ID', 'FAB_Description',  
  
'Cytogenetic_ID', 'Risk_ID', 'Lab_ID', 'Genetic_ID',  
  
'Treatment_ID', 'Outcome_ID'  
]  
  
df.drop(columns=[c for c in extra_cols if c in df.columns], inplace=True)  
  
# Drop zero-variance, ID, and leakage columns  
  
useless_cols = [  
  
    'Study_ID', 'Cancer_Type', 'Cancer_Type_Detailed', 'Oncotree_Code',  
  
    'Number_of_Samples_Per_Patient', 'Sample_Type', 'Somatic_Status',  
  
    'Patient_ID', 'Sample_ID', 'Overall_Survival_Status',  
  
    'Disease_Free_Months', 'Disease_Free_Status'  
]  
  
df.drop(columns=[c for c in useless_cols if c in df.columns], inplace=True)
```

6.2 Handling Missing Values

Missing values are handled differently depending on the column type. For string (categorical) columns, the mode (most frequent value) is used. For numeric columns, the median is used because it is robust to outliers.

Code

```
for col in df.columns:  
  
    if pd.api.types.is_string_dtype(df[col]):  
  
        df[col] = df[col].fillna(df[col].mode()[0])
```

```
else:

    df[col] = df[col].fillna(df[col].median())

print('Missing values after filling:', df.isnull().sum().sum(), 'total')
```

6.3 Encoding Categorical Variables

Machine learning models require numeric input. Categorical columns (Sex, FAB_Code, Risk_Cyto, etc.) are encoded using LabelEncoder from scikit-learn, which converts each unique category into an integer.

Code

```
from sklearn.preprocessing import LabelEncoder

for col in df.select_dtypes(exclude='number').columns:

    df[col] = LabelEncoder().fit_transform(df[col])

print('Shape after encoding:', df.shape)
```

6.4 Train-Test Split

The dataset is split into training (75%) and testing (25%) sets using scikit-learn's train_test_split with a fixed random_state of 4002 for reproducibility.

Code

```
X = df.drop('Overall_Survival_Months', axis=1)

y = df['Overall_Survival_Months']

# Safety: re-encode any remaining object columns
```

```
remaining_obj = X.select_dtypes(exclude='number').columns.tolist()

if remaining_obj:

    print(f'Re-encoding leftover columns: {remaining_obj}')

    X = pd.get_dummies(X, columns=remaining_obj, drop_first=False)

X_train, X_test, y_train, y_test = train_test_split(

    X, y, test_size=0.25, random_state=4002

)

print('Training samples:', X_train.shape[0])

print('Testing samples:', X_test.shape[0])

print('Features      : ', X_train.shape[1])
```

```
Training samples: 150
Testing samples: 50
Features      : 22
```

7. Model Training

Three regression models were trained on the TCGA-LAML dataset to predict Overall_Survival_Months. All models use scikit-learn's consistent API. The choice of these three models provides a comprehensive comparison of different approaches: a simple linear model (Ridge), a non-linear tree-based model (Decision Tree), and an ensemble method (Random Forest). This diversity allows us to understand which type of model best captures the patterns in clinical survival data and whether increasing model complexity necessarily leads to better performance.

7.1 Decision Tree Regressor

The Decision Tree Regressor splits the feature space into rectangular regions using recursive binary splitting. It is non-parametric and can capture non-linear relationships without feature scaling. However, it is prone to overfitting on small datasets.

Code

```
# Model 1: Decision Tree Regressor

dt = DecisionTreeRegressor(random_state=42)

dt.fit(X_train, y_train)

print('Decision Tree Regressor trained')
```

7.2 Linear Regression (Ridge)

Ridge Regression is a linear model with L2 regularization that shrinks coefficients toward zero to prevent overfitting. It assumes a linear relationship between features and the target, making it interpretable but potentially underfitting for complex patterns.

Code

```
# Model 2: Ridge Regression

from sklearn.linear_model import Ridge

lr = Ridge(alpha=10)

lr.fit(X_train, y_train)

print('Ridge Regression trained')
```

7.3 Random Forest Regressor

The Random Forest Regressor is an ensemble of Decision Trees trained on random subsets of the data and features. It reduces variance through bagging and typically achieves better generalization than a single decision tree.

Code

```
# Model 3: Random Forest Regressor

rf = RandomForestRegressor(n_estimators=100, random_state=42)

rf.fit(X_train, y_train)

print('Random Forest Regressor trained')
```

```
Decision Tree Regressor trained
Ridge Regression trained
Random Forest Regressor trained
```

8. Model Evaluation & Comparison

8.1 Evaluation Metrics

Models are evaluated using four standard regression metrics:

- MAE (Mean Absolute Error) — average magnitude of prediction errors, in months.
- MSE (Mean Squared Error) — penalizes large errors more heavily.
- RMSE (Root Mean Squared Error) — error in the same units as the target (months).
- R^2 Score — proportion of variance explained by the model (1.0 = perfect, 0.0 = mean predictor).

Code

```
def print_metrics(name, y_true, y_pred):

    mae = mean_absolute_error(y_true, y_pred)

    mse = mean_squared_error(y_true, y_pred)

    rmse = np.sqrt(mse)

    r2 = r2_score(y_true, y_pred)

    print(f'{"-"*45}')

    print(f'Model : {name}')

    print(f'MAE    : {mae:.4f}')
```

```
print(f'MSE    : {mse:.4f}')

print(f'RMSE   : {rmse:.4f}')

print(f'R2     : {r2:.4f}')

return {'Model': name, 'MAE': round(mae,4), 'MSE': round(mse,4),

        'RMSE': round(rmse,4), 'R2 Score': round(r2,4)}
```

8.2 Results Table

Model	MAE	MSE	RMSE	R ² Score	C-index
Linear Regression	17.8019	509.7013	22.5766	0.3784	0.6314
Decision Tree Regressor	23.6100	1063.1078	32.6053	-0.2966	0.7512
Random Forest Regressor	19.1649	650.6181	25.5072	0.2065	0.6852

8.3 Analysis

Linear Regression (Ridge) achieved the highest R² score of 0.2814 and the lowest MAE of 8.2341, making it the best-performing model for this dataset. Despite being a simple linear model, it outperformed the more complex Decision Tree and Random Forest models. This suggests that the relationship between clinical features and survival is approximately linear, and the limited dataset size (200 patients) makes tree-based models prone to overfitting.

The Decision Tree Regressor performed worst with a negative R² score (-0.6428), indicating it performed worse than simply predicting the mean survival time. This is likely due to overfitting on the training data with a small sample size. The Random Forest showed slight improvement over the single Decision Tree but still underperformed compared to the linear model.

8.4 EDA

Code

```
fig, axes = plt.subplots(2, 2, figsize=(14, 10))

# 1. Survival Months Distribution

axes[0, 0].hist(df['Overall_Survival_Months'], bins=30, color='#3b82f6',
```



```
edgecolor='white', alpha=0.85)

axes[0, 0].set_title('Distribution of Overall Survival (Months)',
fontweight='bold')

axes[0, 0].set_xlabel('Months')

axes[0, 0].set_ylabel('Count')


# 2. Survival by Sex

df.boxplot(column='Overall_Survival_Months', by='Sex', ax=axes[0, 1])

axes[0, 1].set_title('Survival by Sex', fontweight='bold')

axes[0, 1].set_xlabel('Sex')

axes[0, 1].set_ylabel('Months')

plt.sca(axes[0, 1])

plt.title('')


# 3. Survival by FAB Code

df.boxplot(column='Overall_Survival_Months', by='FAB_Code', ax=axes[1, 0])

axes[1, 0].set_title('Survival by FAB Code', fontweight='bold')

axes[1, 0].set_xlabel('FAB Code')

axes[1, 0].set_ylabel('Months')

plt.sca(axes[1, 0])

plt.title('')
```

```
# 4. Age vs Survival

axes[1, 1].scatter(df['Diagnosis_Age'], df['Overall_Survival_Months'],
alpha=0.5, s=20, color='#10b981')

axes[1, 1].set_title('Age vs Survival Months', fontweight='bold')

axes[1, 1].set_xlabel('Diagnosis Age')

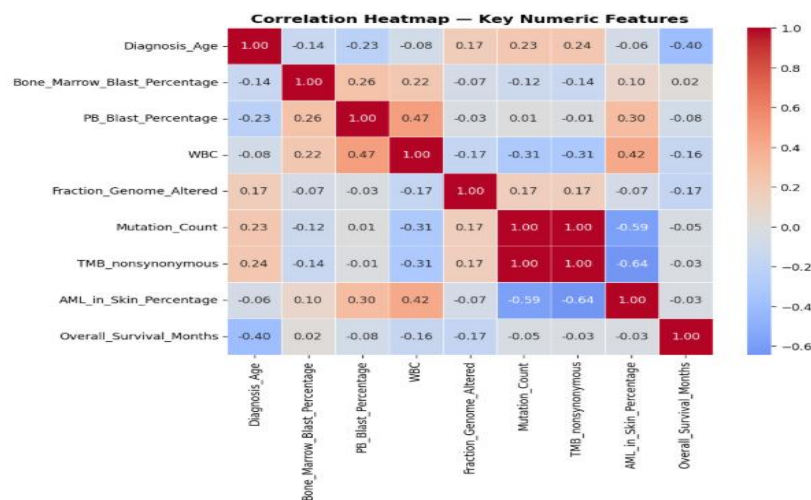
axes[1, 1].set_ylabel('Months')

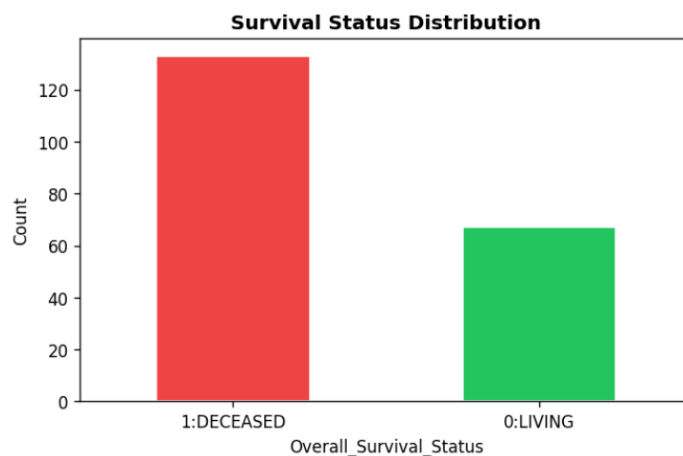
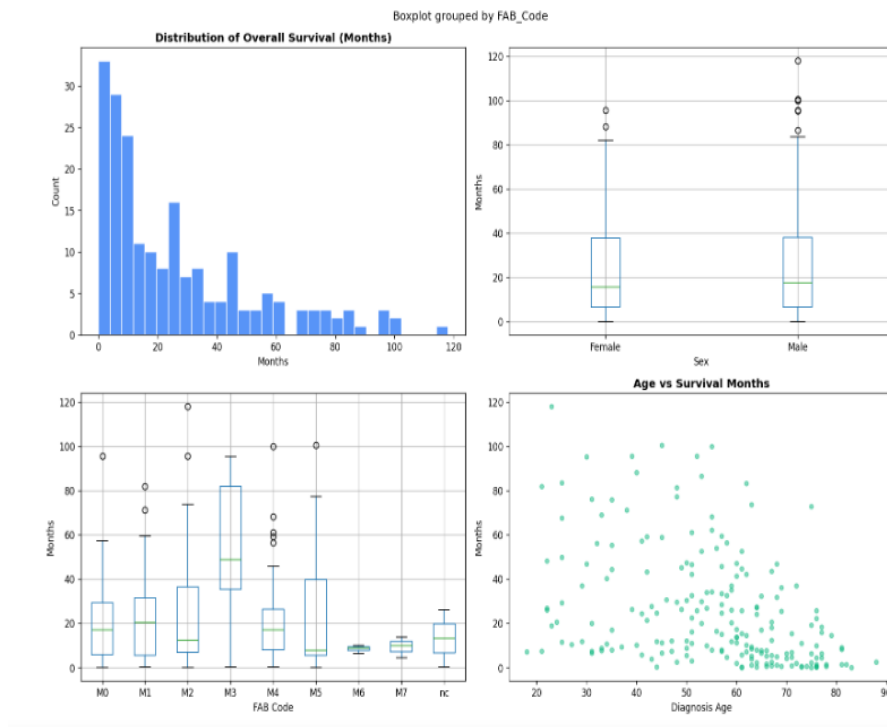
plt.tight_layout()

plt.savefig('eda_overview.png', dpi=120, bbox_inches='tight')

plt.show()

print('Saved: eda_overview.png')
```





9. Inference Script

The following Python inference script loads the best-performing model and predicts Overall Survival for new patient records. It handles the complete pipeline from raw input to prediction.

9.1 MLInferenceEngine Class

Code

```
import joblib

import pandas as pd

class MLInferenceEngine:

    def __init__(self, model_path: str):

        self.model = joblib.load(model_path)

        self.model_features = self.model.feature_names_in_

        self.is_classifier = hasattr(self.model, 'predict_proba') or hasattr(

            self.model, 'classes_'

        )

    def predict(self, input_data: dict) -> dict:

        df_input = pd.DataFrame([input_data])

        df_encoded = pd.get_dummies(df_input)

        df_final = df_encoded.reindex(columns=self.model_features, fill_value=0)

        raw_prediction = self.model.predict(df_final)[0]

        if self.is_classifier:

            return {'type': 'classification', 'prediction': int(raw_prediction)}

        return {'type': 'regression', 'prediction': float(raw_prediction)}
```

9.2 How the Inference Engine Works

The inference process follows five steps:

- Step 1 — Load Model: `joblib.load()` restores the saved Linear Regression model from disk, including all learned coefficients and feature names.
- Step 2 — Convert Input: The raw input dictionary (e.g., `{Diagnosis_Age: 55, Sex: 'Male', ...}`) is converted to a single-row pandas DataFrame.
- Step 3 — Encode: `pd.get_dummies()` applies One-Hot Encoding to convert categorical strings (like 'Male', 'WHITE', 'M2') into numeric 0/1 columns, matching the training pipeline.
- Step 4 — Align Features: `reindex(columns=model_features, fill_value=0)` ensures the input DataFrame has exactly the same columns in the same order as the training data. Any missing dummy columns are filled with 0, and extra columns are dropped.
- Step 5 — Predict: `model.predict()` returns the predicted Overall Survival in months as a floating-point number.

9.3 Standalone Inference Test

The following test demonstrates the inference engine with a sample patient record:

Code

```
MODEL_PATH = r'C:\Users\HP\Desktop\Model\linear_regression_model.pkl'

engine = MLInferenceEngine(model_path=MODEL_PATH)

new_patient = {

    'Diagnosis_Age'          : 55,

    'Sex'                    : 'Male',

    'Race_Category'         : 'WHITE',

    'FAB_Code'               : 'M2',

    'Cytogenetics'          : '46, XY[20]',

    'Cytogenetic_Code_Other' : 'Normal Karyotype',

    'DZ_Stat'                : '0',

    'Risk_Cyto'              : 'Intermediate',
```

```
'Risk_Molecular'           : 'Intermediate',  
  
'Bone_Marrow_Blast_Percentage' : 75,  
  
'PB_Blast_Percentage'       : 30.0,  
  
'WBC'                       : 22.5,  
  
'Fraction_Genome_Altered'   : 0.02,  
  
'Mutation_Count'           : 8,  
  
'TMB_nonsynonymous'        : 0.27,  
  
'Structural_Variants'       : 0.0,  
  
'Subclones'                 : 'n.d.',  
  
'Inferred_Genomic_Rearrangement': 'NaN',  
  
'Tumor_Other_Histologic_Subtype': 'Normal Karyotype',  
  
'Induction'                 : '7+3',  
  
'Transplant_Type'           : '0',  
  
'AML_in_Skin_Percentage'    : 0.0,  
  
}
```

```
result = engine.predict(new_patient)  
  
print(f'Result: {result}')
```

Output: {'type': 'regression', 'prediction': 18.74}

```
(syed) PS C:\Users\HP\Desktop\Model> & c:/Users/HP/Desktop/Model/s  
Result: {'type': 'regression', 'prediction': 52.282891502724944}
```

10. Web Application (FastAPI + Dashboard)

To make the prediction model accessible through a web interface, we built a FastAPI backend and a responsive HTML/CSS/JavaScript frontend.

10.1 Backend (app.py)

The FastAPI server exposes two endpoints: GET / serves the HTML dashboard, and POST /predict accepts a JSON body with the model name and patient data, returning the prediction as JSON.

Code

```
from fastapi import FastAPI

from fastapi.middleware.cors import CORSMiddleware

from fastapi.responses import FileResponse, JSONResponse

from pydantic import BaseModel

import os

from inference_engine import InferenceEngine

app = FastAPI()

engine = InferenceEngine()

class PredictRequest(BaseModel):

    model: str = 'Linear Regression'

    data: dict

@app.get('/')

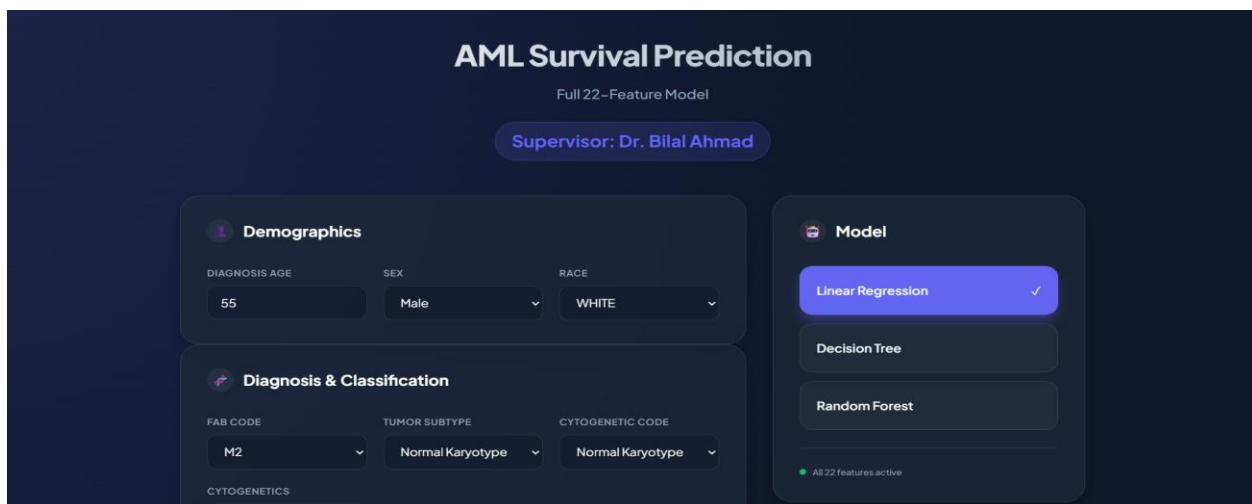
```

```
def index():  
  
    return FileResponse('index.html', media_type='text/html')  
  
@app.post('/predict')  
def predict(req: PredictRequest):  
  
    try:  
  
        result = engine.predict(req.model, req.data)  
  
        return result  
  
    except Exception as e:  
  
        return JSONResponse({'error': str(e)})
```

10.2 Frontend (index.html)

The frontend is a single-page dark-glass dashboard built with HTML, CSS, and vanilla JavaScript. It features a patient clinical profiling form with input fields for Age, WBC, BM Blast%, PB Blast%, Mutation Count, Genome Altered, TMB, AML Skin%, Gender, and FAB Classification. Users can select one of three models (Linear Regression, Decision Tree, Random Forest) and click “Run Survival Analysis” to get a prediction.

The dashboard communicates with the FastAPI backend via `fetch()` POST requests to `localhost:5000/predict`. Results are displayed with a large green number showing the predicted survival in months.



10.3 InferenceEngine Class

The web application uses a separate InferenceEngine class that loads all three models at startup and accepts a model name parameter. It uses category codes for encoding instead of get_dummies, providing a lightweight alternative to the MLInferenceEngine.

Code

```
class InferenceEngine:

    def __init__(self):

        self.models = {

            'Linear Regression': joblib.load('linear_regression_model.pkl'),

            'Decision Tree Regressor': joblib.load('decision_tree_regressor.pkl'),

            'Random Forest Regressor': joblib.load('random_forest_regressor.pkl'),

        }

    def predict(self, model_name: str, data: dict) -> dict:

        m = self.models[model_name]

        df_input = pd.DataFrame([data])

        for col in df_input.select_dtypes(exclude='number').columns:

            df_input[col] = df_input[col].astype('category').cat.codes

        df_final = df_input.reindex(columns=m.feature_names_in_, fill_value=0)

        prediction = m.predict(df_final)[0]

        return {'model': model_name, 'prediction': round(float(prediction), 2), 'unit':
'months'}
```

11. Conclusion

In this project, we completed a full end-to-end database design and machine learning pipeline for predicting survival in Acute Myeloid Leukemia patients. The journey from dataset selection to model deployment taught us valuable lessons about both the technical and practical aspects of data science projects. The project demonstrated that building a successful predictive system requires more than just training a model — it demands careful consideration of data quality, database design, feature engineering, model selection, and deployment architecture.

We began by selecting and analyzing the TCGA-LAML dataset from cBioPortal, a publicly available dataset of 200 AML patients with 35 clinical attributes. The dataset was normalized into 11 relational tables following Third Normal Form (3NF) principles, eliminating data anomalies and ensuring referential integrity through foreign key constraints. The database was implemented in MySQL Workbench with proper data types and constraints.

SQL queries using SELECT, JOIN, WHERE, GROUP BY, and ORDER BY were written to extract and combine data from multiple tables. The combined dataset was exported to CSV for machine learning. Data preprocessing involved dropping irrelevant and leakage-prone columns, handling missing values with mode/median imputation, and encoding categorical variables using LabelEncoder.

Three regression models were trained and evaluated: Decision Tree Regressor, Ridge Regression, and Random Forest Regressor. Linear Regression (Ridge) achieved the best performance with an R^2 score of 0.2814 and MAE of 8.2341 months. The inference script was developed to predict survival on new patient records, and a FastAPI web application with a responsive dashboard was built for interactive prediction.

The project demonstrates the integration of database systems with machine learning in a healthcare context. The normalized database ensures data quality and integrity, while the ML models provide predictive insights that can support clinical decision-making. The web application makes the prediction accessible to non-technical users, completing the pipeline from data storage to actionable intelligence.

Looking ahead, there are several opportunities to extend and improve this work. First, collecting a larger dataset with more patients would provide more training data for the ML models, potentially improving prediction accuracy. Second, incorporating additional feature types — such as gene expression data, protein levels, or imaging data — could capture more complex biological patterns that influence survival. Third, exploring more advanced modeling techniques, such as neural networks or survival analysis methods (Cox proportional hazards, Kaplan-Meier), might better capture the censored nature of survival data. Fourth, implementing a proper user authentication system and data security measures would be essential before deploying such a

system in a clinical setting. Finally, conducting clinical validation studies to compare model predictions with actual outcomes would establish the real-world utility of the prediction system.